ПиКЯП
Отчёт по РК1.
Новикова Яна, ИУ5-35Б
Вариант: 26

Цель работы:

Изучить возможности языка Python для работы с данными: создание классов, связанных отношениями один-ко-многим и многие-ко-многим, и выполнение запросов к ним с использованием list/dict comprehensions и функций высших порядков (sorted, max, any).

Описание классов:

Предметная область (вариант 26): «Студенческая группа» – «Учебный курс».

1. StudentGroup («Студенческая группа») – сторона «много»: id – идентификатор группы; name – название группы; student_count – количество студентов (количественный признак, нужен для запроса Г2); course_id – идентификатор курса (связь один-ко-многим).

2. Course («Учебный курс») – сторона «один»: id – идентификатор курса; name – название учебного курса.

3. GroupCourse («Группы и курсы») – класс для связи многие-ко-многим: group_id – идентификатор группы; course_id – идентификатор курса.

Код:

```
from operator import itemgetter


class StudentGroup:
    def __init__(self, id, name, student_count, course_id):
        self.id = id
        self.name = name
        self.student_count = student_count
        self.course_id = course_id


class Course:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class GroupCourse:
    def __init__(self, group_id, course_id):
        self.group_id = group_id
        self.course_id = course_id
```

```
courses = [
    Course(1, 'Алгоритмы'),
    Course(2, 'Базы данных'),
    Course(3, 'Python'),
    Course(4, 'Компьютерные сети'),
    Course(5, 'Архитектура компьютеров'),
]

groups = [
    StudentGroup(101, 'ИУ5-31Б', 25, 1),
    StudentGroup(102, 'ИУ5-32Б', 28, 1),
    StudentGroup(103, 'ИУ5-33Б', 22, 2),
    StudentGroup(104, 'ИУ5-34Б', 30, 3),
    StudentGroup(105, 'ИУ5-35Б', 26, 5),
]

groups_courses = [
    GroupCourse(101, 1),
    GroupCourse(102, 1),
    GroupCourse(104, 1),
    GroupCourse(101, 2),
    GroupCourse(103, 2),
    GroupCourse(105, 2),
    GroupCourse(102, 3),
    GroupCourse(103, 3),
    GroupCourse(104, 3),
    GroupCourse(101, 4),
    GroupCourse(105, 4),
    GroupCourse(103, 5),
    GroupCourse(104, 5),
    GroupCourse(105, 5),
]

def main():
    one_to_many = [(g.name, g.student_count, c.name)
                   for c in courses
                   for g in groups
                   if g.course_id == c.id]

    many_to_many_temp = [(c.name, gc.course_id, gc.group_id)
                         for c in courses
                         for gc in groups_courses
                         if c.id == gc.course_id]

    many_to_many = [(g.name, g.student_count, course_name)
                    for course_name, course_id, group_id in many_to_many_temp
                    for g in groups if g.id == group_id]
```

```
    print('Задание Г1')
    res_1 = {c.name: [g for g, _, cn in one_to_many if cn == c.name]
             for c in courses
             if c.name.startswith('A') and any(cn == c.name for _, _, cn in one_to_many)}
    print(res_1)

    print('\\nЗадание Г2')
    res_2_unsorted = []
    for c in courses:
        c_counts = [count for _, count, cn in one_to_many if cn == c.name]
        if c_counts:
            res_2_unsorted.append((c.name, max(c_counts)))
    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    print(res_2)

    print('\\nЗадание Г3')
    res_3 = sorted(many_to_many, key=itemgetter(2))
    print(res_3)


if __name__ == '__main__':
    main()
```

Результат:

Задание Г1

{'Алгоритмы': ['ИУ5-31Б', 'ИУ5-32Б'], 'Архитектура компьютеров': ['ИУ5-35Б']}

Задание Г2

[('Python', 30), ('Алгоритмы', 28), ('Архитектура компьютеров', 26), ('Базы данных', 22)]

Задание Г3

[('ИУ5-32Б', 28, 'Python'), ('ИУ5-33Б', 22, 'Python'), ('ИУ5-34Б', 30, 'Python'),
('ИУ5-31Б', 25, 'Алгоритмы'), ('ИУ5-32Б', 28, 'Алгоритмы'), ('ИУ5-34Б', 30, 'Алгоритмы'),
('ИУ5-33Б', 22, 'Архитектура компьютеров'), ('ИУ5-34Б', 30, 'Архитектура компьютеров'),
('ИУ5-35Б', 26, 'Архитектура компьютеров'), ('ИУ5-31Б', 25, 'Базы данных'),
('ИУ5-33Б', 22, 'Базы данных'), ('ИУ5-35Б', 26, 'Базы данных'),
('ИУ5-31Б', 25, 'Компьютерные сети'), ('ИУ5-35Б', 26, 'Компьютерные сети')]

Вывод:

В ходе рубежного контроля были созданы классы «Студенческая группа» и «Учебный курс», связанные отношениями один-ко-многим и многие-ко-многим, и списки тестовых данных к ним. Реализованы три запроса варианта 26: вывод курсов, название которых начинается с буквы «А», вместе с их группами (Г1); вывод курсов с максимальным количеством студентов в связанных группах, отсортированный по этому значению (Г2); вывод всех связанных групп и курсов по связи многие-ко-многим, отсортированный по курсам (Г3). Запросы реализованы с помощью list/dict comprehensions и функций высших порядков, результаты выполнения программы соответствуют условиям заданий.